DEBRECENI EGYETEM
INFORMATIKAI KAR

SZAKDOLGOZAT

Virtuális ruhásszekrény

Témavezető:
Major Sándor Roland
Tanársegéd

Készítette:
Nagy Szabolcs
Programtervező informatikus

DEBRECEN
2025

1. Tartalomjegyzék

2. Bevezetés *5*

3. Alkalmazott technológiák és eszközök rövid ismertetése *7*

3.1. React.js **7**

3.1.1. Főbb jellemzők 7

3.1.2. HTML, CSS, JS 7

3.2. Spring Boot **8**

3.2.1. Főbb jellemzők 8

3.2.2. Felhasznált függőségek 9

3.3. MySQL **9**

3.4. Axios **9**

3.4.1. Főbb jellemzők 10

3.5. GitHub **10**

4. Az alkalmazás felépítése *11*

4.1. Frontend **11**

4.1.1. Bejelentekezés és regisztráció 11

4.1.1.1. A regisztráció folyamata 11

4.1.1.2. A bejelentkezés folyamata 12

4.1.1.3. Felhasználói visszajelzések, rendszerüzenetek 12

4.1.2. Öltözetek 12

4.1.2.1. Az oldal felépítése 13

4.1.3. Tervező 13

4.1.4. Galéria 15

4.1.4.1. Részletes nézet 15

4.1.4.2. Szűrés és keresés 16

4.1.4.3. Új ruhadarab feltöltésének menete 16

4.2. Backend **17**

4.2.1. Az adatbázis 18

4.2.1.1. Az item tábla 19

4.2.1.2. Az outfit tábla 19

4.2.1.3. A user tábla 20

4.3. A Data réteg 20

4.3.1. Entitás osztályok 21

4.3.1.1. Gyakran előforduló annotációk 21

4.3.1.2. UserEntity 21

4.3.1.3. ItemEntity 22

4.3.2. Repository 22

4.4. A Service réteg 22

4.4.1. DTO (Data Transfer Object) 23

4.4.1.1. Felépítésük 23

4.4.1.2. Megszorítások 23

4.4.2. Service osztályok 24

4.4.2.1. Metódusok általános logikája 25

4.4.2.2. AuthService 26

4.4.2.3. JwtAuthService 27

4.4.2.4. UserService 28

4.4.2.5. ItemService 28

4.4.2.6. OutfitService 30

4.5. A Controller réteg 31

4.5.1. Gyakran előforduló annotációk 31

4.5.2. AuthController 31

4.5.2.1. Regisztráció, belépés, kijelentkezés 31

4.5.2.2. Hitelesítés állapota 32

4.5.3. ItemController 32

4.5.3.1. @PostMapping("/items") 33

4.5.3.2. @GetMapping("/items") 33

4.5.3.3. @DeleteMapping("/items/{id}") 33

4.5.3.4. Képek kezelése 33

4.5.4. OutfitController 33

4.6. Biztonság, hitelesítés és egyéb konfigurációk 34

4.6.1. JWT (JSON Web Token) alapú hitelesítés 34

4.6.1.1. Fontos lépések 34

4.6.2. Spring Security 35

4.6.2.1. A konfigurációs fájl 36

4.6.2.2. CORS (Cross-Origin Resource Sharing) 36

5. Lehetőségek az alkalmazás továbbfejlesztésére 37

5.1. Automatikus öltözetajánlás..... 37

5.2. Képfelismerés a ruhadarabok azonosítására 37

5.3. Naptár integráció..... 37

5.4. Időjárás előrejelzés 37

5.5. Indexelés az adatbázisban 37

6. Összefoglalás 38

7. Irodalomjegyzék 40

8. Köszönetnyilvánítás..... 43

2. Bevezetés

Szakdolgozatom fő célja, hogy egy mindennapos tevékenység, mely egész életünkön végigkísér és összességében rengeteg időt felemészthet, megkönnyítésére ajánljak egy lehetőséget. Minden ember életében valószínűleg legalább egyszer előfordult már, hogy a „Mit vegyek fel a holnapi fontos eseményre?” kérdés megválaszolásával nem időben foglalkozott, így az útra kelés előtt percekkel derült ki, hogy a betervezett ruhadarab túl szűk. Sajnos ez egy olyan folyamatot eredményez az esetek nagy többségében, amely a nap hátralévő részére jelentősen befolyásolja az ember hangulatát, természetesen ritkán pozitív irányba. Erre a könnyen elkerülhető problémára szeretnék megoldást nyújtani egy virtuális ruhásszekrény keretein belül, mely keretin belül könnyebben megtervezhetővé szeretném varázsolni az öltözködést.

Mai világunkban már rengeteg dolog elérhető pár kattintással, gombnyomással, sőt akár kimondott szavakkal is. Érdekünk, hogy egyes kevésbé fontos tevékenységek időigényét a lehető legjobban minimalizáljuk, így több időt nyerve minden más fontosabbnak, mint például a munka, a tanulás vagy akár az alvás. Az időgazdálkodás javítása elengedhetetlen a rohanó világunkban. Hosszas kutatás után sajnálattal kellett rájönnöm, hogy az öltözködésnél eddig kevesen gondoltak arra, hogy ideje felhagyni a hagyományos módszerekkel.

A ruházataink megfelelő kezelése nemcsak praktikus, hanem hozzájárulhat a fenntarthatósághoz is. Egy rendszerezett és áttekinthető ruhatár segítségével elkerülhetőek a felesleges vásárlások, ami nem csak környezetvédelmi, hanem anyagi szempontból is jelentős. Egy digitális ruhásszekrény véleményem szerint elősegíti, hogy a felhasználók tudatosabb döntéseket hozzanak a ruházatuk terén.

Természetesen léteznek már alkalmazások, amelyek lehetőséget biztosítanak ruhadarabjaink online tárolására, viszont olyan általam fontosnak vélt funkciók hiányoznak ezekből, amik véleményem szerint hozzájárulnának a mindennapok megkönnyítéséhez.

Célom így nem csak egy működő applikáció létrehozása volt, hanem konkrét tervekkel és szigorú elvárásokkal készítettem munkámat: egy felhasználóbarát, egyszerű, de mégis minden létfontosságú feladatot ellátó webes alkalmazást szerettem volna. Fontosnak tartottam, hogy az felöltözést megelőző minden lépés (tervezés, válogatás) egyszerűen, gyorsan és akár napokkal előre elvégezhető legyen az oldalon keresztül, ezzel gyorsítva ezt a folyamatot. Emellett különös figyelmet fordítottam arra, hogy átláthatóbb legyen, mint egy hagyományos ruhatár: minden fontos funkció egy helyen, egy-két kattintással elérhető legyen, illetve csak és kizárólag

azt lássa a felhasználó, amit tényleg szeretne. Míg egy szekrénynél előfordulhat, hogy több hasonló ruhadarabot is ki kell vennünk, hogy egy konkrétat megtaláljunk, addig az alkalmazásban elegendő egy egyszerű szűrő (például szín, márka, típus) vagy kereső (például bármilyen kulcsszó, mellyel beazonosítható egy darab) használata.

Ugyan az online térre váltás egy ruhatárnál egy új lépést vonz magával, mely eleinte időigényesnek tűnhet. Ez nem más, mint a képek feltöltése minden egyes éppen birtokolt ruhadarabunkról a weboldalra, amelyet szeretnénk ott is viszont látni. Ezen folyamat időigényét a lehetőségekhez mérten próbáltam a lehető legjobban csökkenteni, az ehhez használt módszereket az „Az alkalmazás felépítése" szakaszban részletezem.

Alkalmazásom fejlesztéséhez a napjainkban legelterjedtebb technológiákat választottam, amelyekből a három legfontosabb:

- a React.js, a felhasználói felület megvalósításához,
- a MySQL, az adatbázishoz,
- a Spring Boot, a backend megvalósításához.

Napjainkban a felhasználók kimagasló része csak használja az alkalmazásokat és nincs igazán tisztában vele, hogy hogyan is működik az egész a háttérben. Dolgozatommal szeretném elérni, hogy az olvasó megértse alkalmazásom működését, illetve, hogy betekintést nyerhessen a háttérben zajló folyamatokra és azok felépítésére. Ugyan a háttéri részek egy ebben a világban nem jártas olvasónak vélhetően nem túlzottan érdekesek, sem informatívak, de a felhasználói felület kezeléséről és működéséről minden fontos részletet bemutatok, így kedvezve mindenkinek.

3. Alkalmazott technológiák és eszközök rövid ismertetése

3.1. React.js

A React [1] egy nyílt forráskódú JavaScript-könyvtár, megjelenése és karbantartottsága 2013 óta a Meta (korábban Facebook) amerikai cég nevéhez köthető. Elsősorban interaktív felhasználói felületek létrehozására szolgál, kimagasló hangsúlyt fektetve az egyoldalas webalkalmazásokra (SPA – Single-page application) [2]. A komponensalapú fejlesztést részesíti előnyben, ahol az alkalmazásunkat modularizált, újra felhasználható elemekből építjük fel. Központi egysége a JSX (JavaScript XML) [3], mely lehetővé teszi a HTML-szerű szintaxis használatát a JavaScript kódon belül. Lényegében a HTML, CSS és JavaScript ötvözete, bár ezen hagyományos technológiákon felül különféle modern optimalizációkat, absztrakciókat, illetve egyéb hasznos funkciókat is biztosít a fejlesztők számára.

3.1.1. Főbb jellemzők

1. **Virtual DOM:** Egy memóriában fenntartott virtuális DOM-ot használ, ezzel optimalizálva annak frissítéseit. A valódi DOM csak azon részei frissülnek, amelyek változáson mentek át, ezzel növelve a teljesítményt.
2. **Komponens alapú felépítés:** A React-tal készült alkalmazások többnyire kisebb, önálló és újra felhasználható komponensekből állnak, amelyek csak egy adott funkciót vagy felhasználói felületet képviselnek.
3. **Hookok:** A 16.8-as verzióban kerültek hozzáadásra a React-hoz, segítségükkel egyszerűbb megoldásokat nyújtva az állapotkezelésre (például a beépített useState és useEffect [4]) és egyéb funkciók használatára, különféle osztályok alkalmazása nélkül.
4. **Külső könyvtárak támogatása:** Széleskörben támogatottak az ilyen fajta könyvtárak, melyek egyes funkciók megvalósítását leegyszerűsítik és hatékonyabbá teszik (például Router – navigációhoz [5], Axios – API hívásokhoz)

3.1.2. HTML, CSS, JS

A HTML (HyperText Markup Language) [6] a weboldalak alapját képzi, meghatározza azok szerkezetét. Egy weboldal egy hierarchikus rendszerben épül fel, ahol az egyes HTML-elemek (például bekezdések, képek, linkek, gombok) egymásba ágyazhatóak. Önmagában ugyan nem ajánlott a használata, mivel semmilyen lehetőséget nem biztosít a vizuális megjelenés személyre szabására.

A vizuális megjelenítés személyre szabásának feladata esik a CSS (Cascading Style Sheets) [7] stílusleíró nyelvre, amely lehetővé teszi a HTML-elemek vizuális formázását (például színek,

betűtípusok, méretek, elhelyezkedések). A CSS-nek köszönhetően van lehetőségünk reszponzív weboldalak kialakítására, biztosítva, hogy a képernyőmérettől függetlenül megfelelően jelenjen meg a kívánt tartalmunk. A React által előnyben részesített komponens alapú felépítés esetén gyakran előfordul a CSS modulok [8] használata, mellyel könnyen elkülöníthető a különféle komponensek stílusa.

A JS (JavaScript) [9] egy magas szintű programozási nyelv, amely interaktív és dinamikus funkciókat biztosít a weboldalak számára, illetve a React gerincét képzi. Lehetőséget ad például eseménykezelésre, űrlapok validálására vagy a felhasználó által elvégzett műveletekre történő reagálásra, mindezt anélkül, hogy az oldalt újra kellene tölteni, így elősegítve a felhasználóbarát élményt.

3.2. Spring Boot

A Java [10] egy rendkívül népszerű, platformfüggetlen nyelv, melynek legnagyobb előnye, hogy a „Write Once, Run Anywhere" elvet követi, azaz a lefordított Java programok bármilyen operációs rendszeren futtathatóak a megfelelő futtatókörnyezettel (JVM – Java Virtual Machine [11]). A Spring Boot [12] egy nyílt forráskódú, Java alapú keretrendszer, amelyet a Spring Framework részeként fejlesztettek, azzal a céllal, hogy leegyszerűsítse a Java alkalmazások fejlesztését, minimalizált konfigurációs lépésekkel, automatizált a beállításokkal és beépített szerverekkel (például Tomcat). Fő célja a fejlesztési folyamat rövidítése különösen a mikroszolgáltatások és webalkalmazások esetén.

3.2.1. Főbb jellemzők

1. **Automatikus konfiguráció:** A Spring Boot automatikusan beállítja az alkalmazásunk konfigurációit, komponenseit a függőségeink alapján.
2. **Beépített szerverek:** A Spring Boot beépített szerverekkel rendelkezik, mint például a Tomcat, Jetty és az Undertow, így nincs szükségünk ezek külső konfigurálására, azaz alkalmazásink önállóan is futtathatóak.
3. **REST API fejlesztés:** Az automatikusan kezelt szerverkonfigurációk, útvonalak és az adatok visszaküldésének köszönhetően rendkívül egyszerű a RESTful API-ok [13] fejlesztése. Elegendő csupán néhány annotáció (például @RestController, @RequestMapping [14]) és néhány sor kód ahhoz, hogy egy teljesértékű végpontot állítsunk elő.
4. **Integráció:** Kifejezetten jól és zökkenésmentesen működik különféle egységek (például MySQL, Spring Security [15]) integrálása. Az adatbázis kezelése például az

integrált JPA-nak (Java Persistence API) [16–17] és a Hibernate-nek [18] köszönhetően rendkívül egyszerű és konzisztens.

Alkalmazásom fejlesztése során használtam továbbá a Maven-t [19], mely egy elengedhetetlen build/projektmenedzser eszköz. Ez az eszköz egy pom.xml nevezetű fájlban tartalmazza a projektünk konfigurációját, melyben meghatározhatóak különféle függőségek [20], build konfigurációk és egyéb beállítások is.

3.2.2. Felhasznált függőségek

- **`org.projectlombok`:** Annotációkon (például `@Data`, `@NoArgsConstructor`) keresztült csökkenti a boilerplate kódot, ezzel gyorsítva a fejlesztés folyamatát. [21]
- **`org.modelmapper`:** Egyszerűsített megoldást nyújt a Java objektumok (például entitások) és DTO-k közötti konverzióra. [22]
- **`io.jsonwebtoken`:** A JWT (JSON Web Token) generálásáért és annak validálásáért felel, mely elengedhetetlen a felhasználói hitelesítéshez. [23]
- **`spring-boot-starter-validation`**: A kezelt adatok helyességének és formai követelményeknek való megfelelésének ellenőrzését teszi lehetővé annotációkon (például @NotNull, @Email) keresztül. [24]

3.3. MySQL

A MySQL [25] egy nyílt forráskódú, relációs adatbázis-kezelő rendszer. Használata kifejezetten elterjedt a webalkalmazások köreiben különféle adatok strukturált tárolásánál. Szerkezete táblákból áll, melyben sorok és oszlopok találhatóak, az tárolt adatok pedig SQL-lel (Structured Query Language) [26] hozzáférhetőek, kezelhetőek. Webalkalmazások esetében tipikusan a szerveroldali perzisztens adatréteg felépítésére szolgál. Remekül skálázható, támogatja az indexelést és a particionálást, mindezzel tovább növelve a lekérdezések gyorsaságát, illetve platformfüggetlen, így akár Windows, MacOS és Linux rendszereken is remekül alkalmazható.

3.4. Axios

Az Axios [27] egy promise-alapú HTTP [28] kliens, amelyet főként JavaScript-tel karöltve használnak API hívások kezelésére. A technológia kifejezetten elterjedt a React-ot használó környezetekben, hiszen támogatást nyújt az aszinkron műveletekre, illetve automatikusan kezeli a JSON [29] adatkonverziót és egyéb hasznos funkciókra.

3.4.1. Főbb jellemzők

1. **Interceptors:** Lehetőséget nyújtanak a HTTP kérések és válaszok globális módosítására, ezzel lehetőséget nyújtva például az autentikációs token elhelyezésére a fejlécmezőben.
2. **Adatkonverzió:** Mivel az Axios ezt automatikusan elvégzi, így nincs szükség külön `JSON.parse()` vagy `JSON.stringfy()` műveletekre.
3. **Promise-alapú kérések:** Mivel az Axios a JavaScript Promise API-t használja az aszinkron műveletek kezelésére, így lehetőségünk van ezek hatékony kezelésére. Mivel minden kérés egy Promise-t ad vissza, így annak eredményét a `.then()`, vagy hibáját a `.catch()` metódussal kezelhetjük. Ezek akár egymásba is láncolhatóak, de a könnyebb olvashatóság és tisztább kód érdekében használható az async/await páros is.

3.5. GitHub

A GitHub [30] egy széleskörben elterjedt verziókezelő platform, a Git [31] rendszerre épül. Tárolók használatával nyújt lehetőséget a felhasználói számára, hogy saját kódjaikat kezeljék vagy akár együttműködjenek egymással.

Az alkalmazásom kódja az alábbi linken érhető el: https://github.com/szabbi/virtual-wardrobe

3.6. Mesterséges intelligencia

Szakdolgozatom készítése során többször is használtam mesterséges intelligenciát (ChatGPT [32]), főként a monotonabb, kevésbé lényeges feladatok elvégzéséhez. A legjobb példa erre a frontend stílusának kialakítása, amely alatt a színösszeállítást, betűstílusokat és méreteket, illetve az arányok kialakítását értem. Rendkívül hasznos volt, hogy segített egy elfogadhatóbb kinézetet varázsolni az alkalmazásomnak, mivel e feladat elvégzése nem kifejezetten az erősségem.

Emellett többnyire kérdések formájában kértem javaslatokat egyes elemek megtervezéséhez, tökéletesítéséhez, mint például arra vonatkozóan, hogy egyes ruhadarabok milyen kategóriákba sorolhatóak, illetve milyen tulajdonságokkal jellemezhetőek.

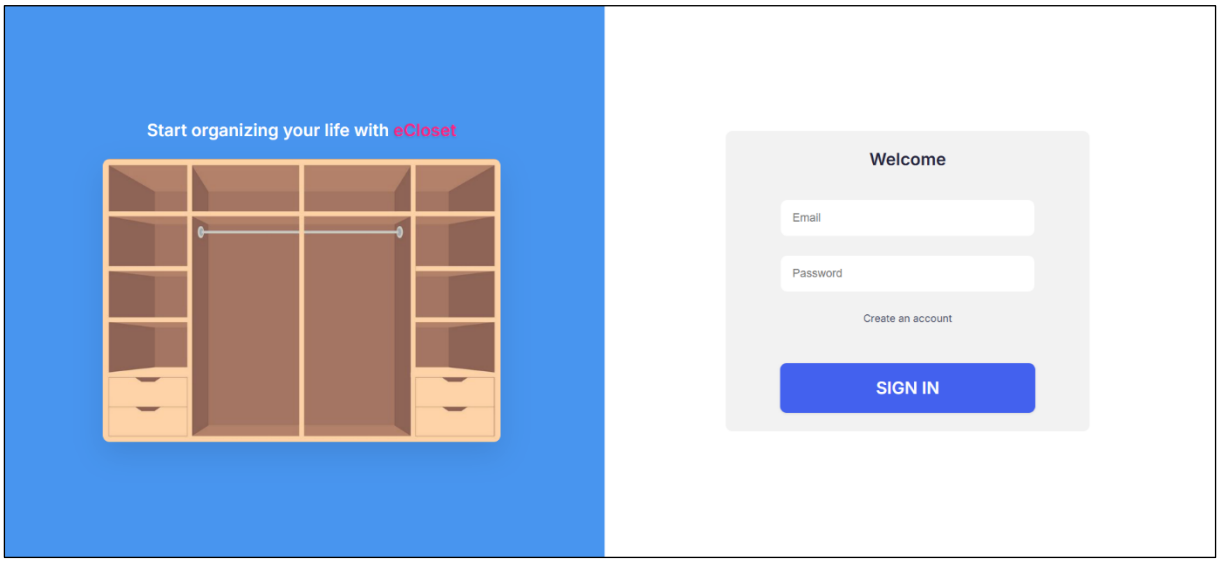
4. Az alkalmazás felépítése

4.1. Frontend

A virtuális ruhatár a felhasználói élményt központban tartva jött létre. A reszponzív és egyszerűen navigálható felületnek, illetve annak letisztultságának köszönhetően mindenki számára könnyen kezelhető a weboldal, így egyszerűsítve az alkalmi vagy akár mindennapi ruhaválasztást a felhasználók számára. A következő pár oldalban a felületen elérhető különböző oldalak és azok funkcióinak részletes ismertetője olvasható.

4.1.1. Bejelentekezés és regisztráció

Mivel maga az öltözködés egy erősen személyre szabott tevékenység, így értelemszerűen egy ruhatárnál szükséges a felhasználói fiókok kezelése. Az weboldal meglátogatásakor az első dolog, ami fogadja minden egyes felhasználót az a bejelentkezést biztosító oldal, ahonnan elérhető a regisztráció is.



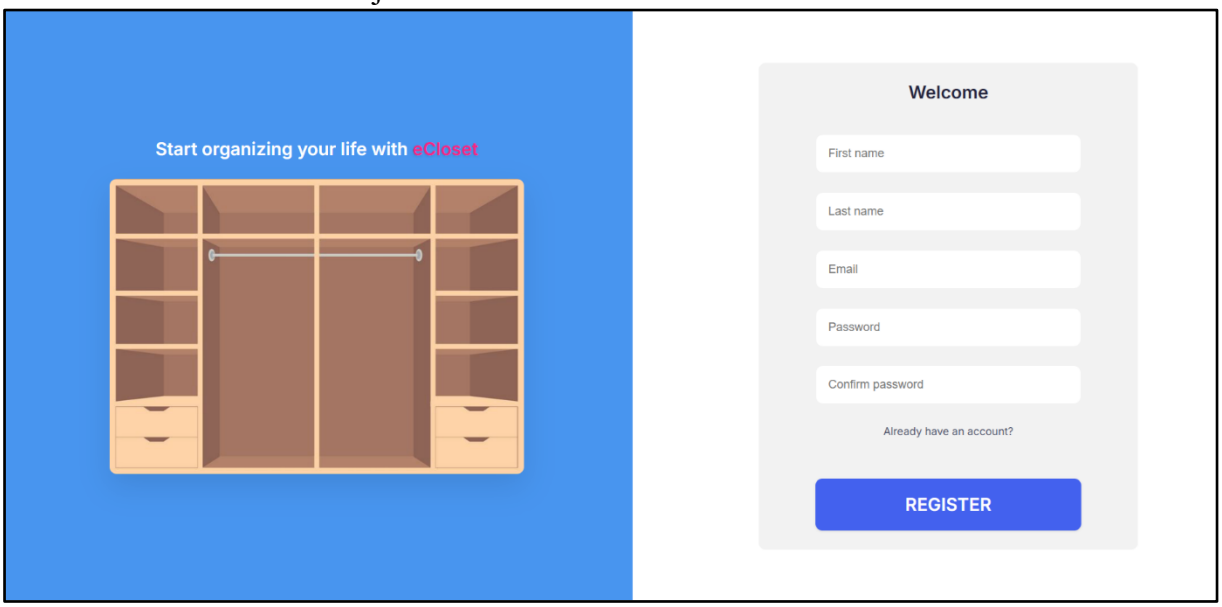
1. ábra – Az alkalmazás bejelentkezésre szolgáló felülete

4.1.1.1. A regisztráció folyamata

Ahhoz, hogy a webalkalmazás használható legyen, először szükséges egy fiók regisztrálása, amely felülete a „Create an account” szövegre kattintva érhető el. Ezáltal a regisztrációs oldalra kerül a felhasználó, ahol a következő adatokat szükséges megadnia:

- vezeték- és keresztnév,
- e-mail cím,
- jelszó,
- jelszó megerősítése.

Fontos, hogy a vezeték- és keresztnevet leszámítva minden mezőben van egy elvárt formátum, mely betartása kötelező, hisz ellenkező esetben nem véglegesíthető a regisztráció sem. Amennyiben már esetlegesen rendelkezünk egy fiókkal az „Already have an account?” feliratra kattintva visszakerülünk a bejelentkezéshez.



2. ábra - Az alkalmazás regisztrációra szolgáló felülete

4.1.1.2. A bejelentkezés folyamata

Sikeres regisztrációt követően, a felhasználó visszakerül az oldalra, ahol az előbbiekben megadott e-mail cím és jelszó kombinációjával már szabadon be tud jelentkezni. A rendszer ellenőrzi az adatok helyességét és amennyiben azok megfelelőek, azaz megtalálhatóak az adatbázisban, a felhasználót belépteti a főoldalra, amely a „Gallery”. Helytelen adatok esetén hibajelzés jelenik meg, amely figyelmezteti a felhasználót a problémára.

4.1.1.3. Felhasználói visszajelzések, rendszerüzenetek

A rendszer minden eseményt követően informatív visszajelzéseket ad a felhasználónak, például:

- „Sikeres bejelentkezés!”
- „Hibás jelszó vagy e-mail cím!”
- „A megadott e-mail cím már használatban van!”
- „Sikeres regisztráció!”

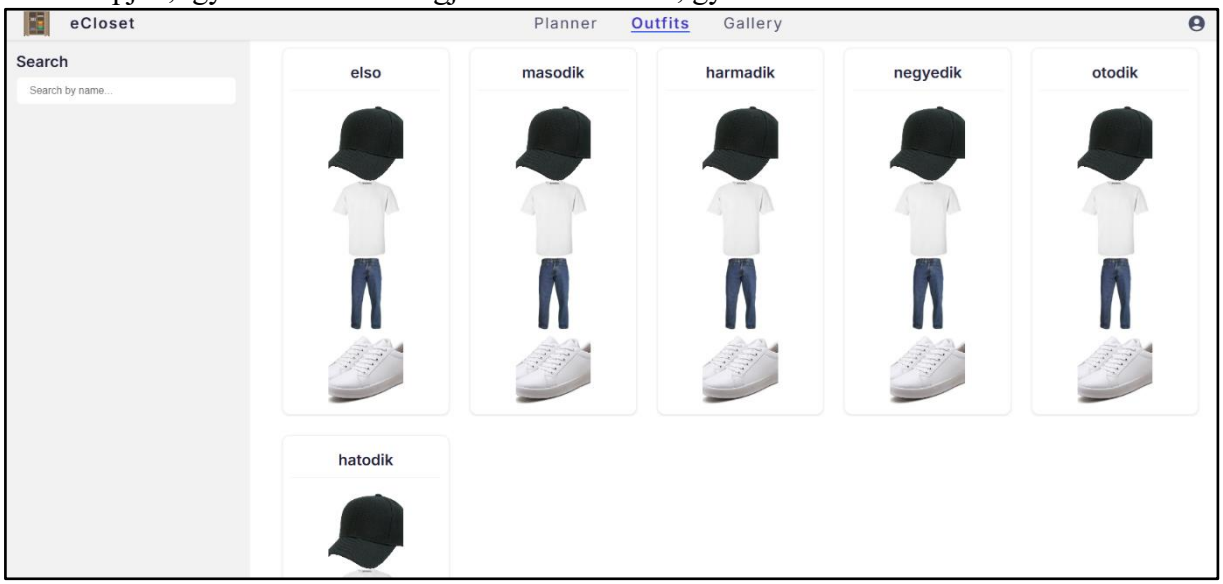
4.1.2. Öltözetek

Mivel a webalkalmazásom elsődleges célja az öltözködés gyorsítása, így úgy gondoltam, hogy szükséges a már elkészített öltözetek külön oldalra szervezése, ezzel gyorsítva azok elérését.

Ez az alkalmazás egyik központi eleme, ahol megtekinthetőek, módosíthatóak vagy akár törölhetőek is az adott elemek.

4.1.2.1. Az oldal felépítése

Központjában a felhasználó által elmentett öltözetek láthatóak, kisebb kártyák formátumában, melyben megtalálható az adott összeállítás neve, illetve ez alatt a kiválasztott ruhadarabok. Az oldal bal oldalán továbbá megtalálható egy kereső, amely segítségével kereshetünk az öltözetek neve alapján, így szűkíthető a megjelenített tartalom, gyorsítva a kiválasztást.



3. ábra - Az alkalmazás öltözetek felülete

Minden egyes összeállítás kártyájára rá lehet kattintani, ezzel megnyitva a hozzá tartozó menüt, amely a 4. ábrán látható. Itt érhetőek el a törlési, illetve szerkesztési gombok is a jobb felső sarokban. Maga a felület nagyobb képeket biztosít a ruhadarabokról, így az jobban szemügyre vehető.

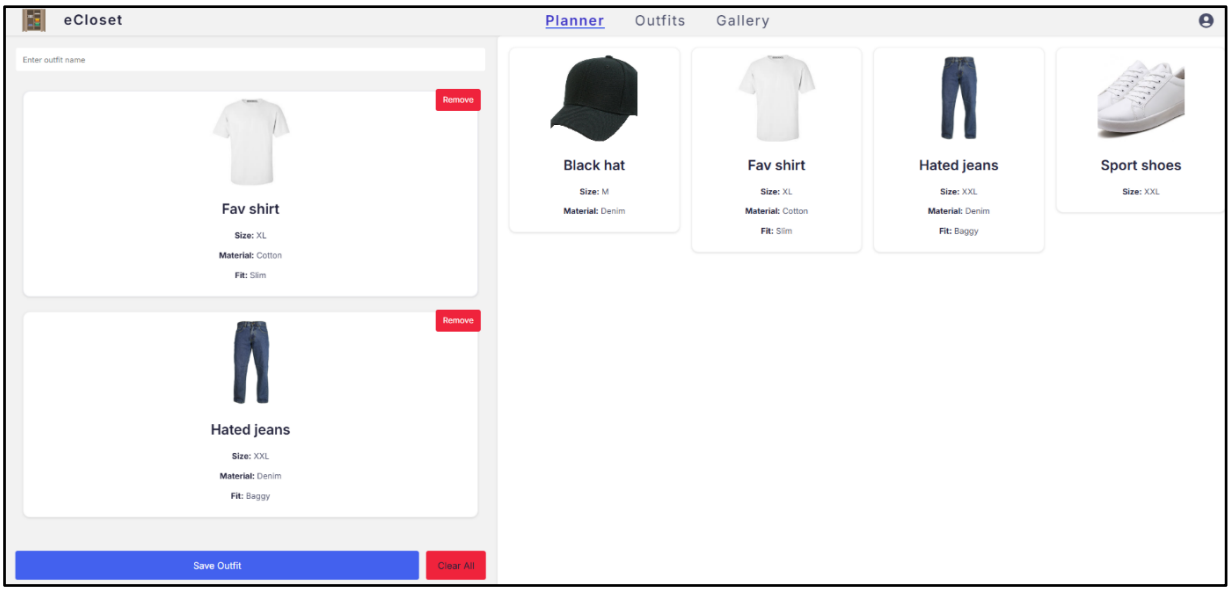
4. ábra - Egy tetszőleges öltözetre való kattintás után felugró ablak

4.1.3. Tervező

Az alábbi oldalon van lehetősége a felhasználóknak új öltözetek összeállítására. A bal oldalon található meg maga a tervezői felület, a jobb oldalon pedig a felöltött ruhák. Egy ruhára kattintva az megjelenik a bal oldalon, a kattintás sorrendjében.

Fontos, hogy egyszerre maximum négy darabot használhatunk fel, viszont alsó korlátozás nincs.

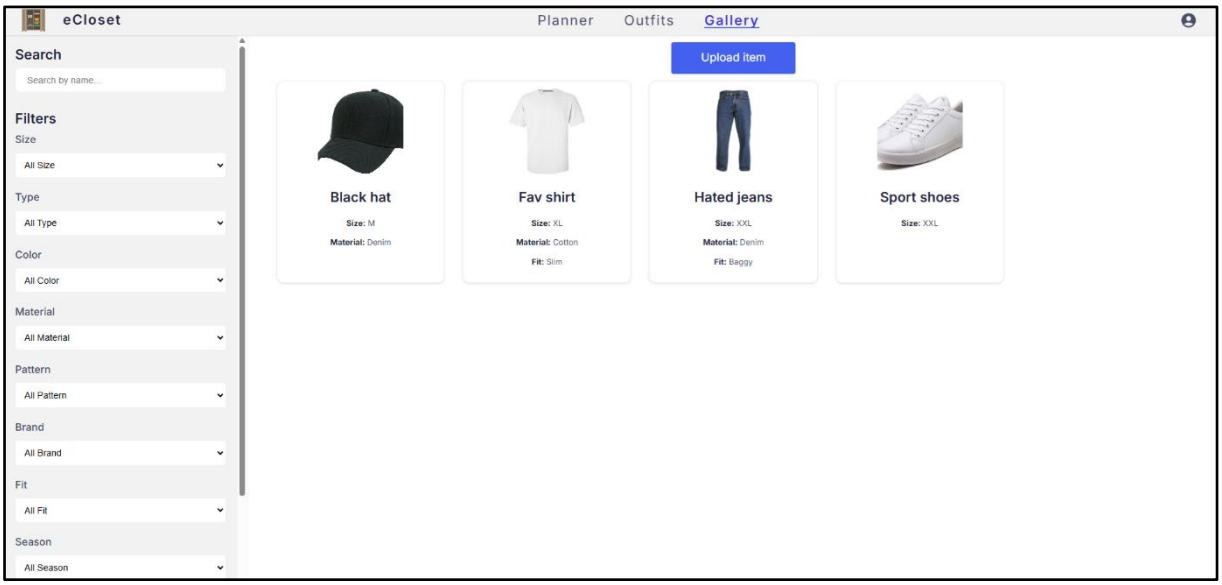
Ammennyiben tévesen kattintottunk egy darabra, az eltávolítható a „Remove” gombra kattintva, vagy akár az egész felület alaphelyzetbe állítható a „Clear All” gomb használatával. A megfelelő darabok kiválasztását, illetve a kívánt név megadását követően a „Save Outfit” gomb használatával menthető el az összeállítás.



5. ábra – Az alkalmazás tervező felülete

4.1.4. Galéria

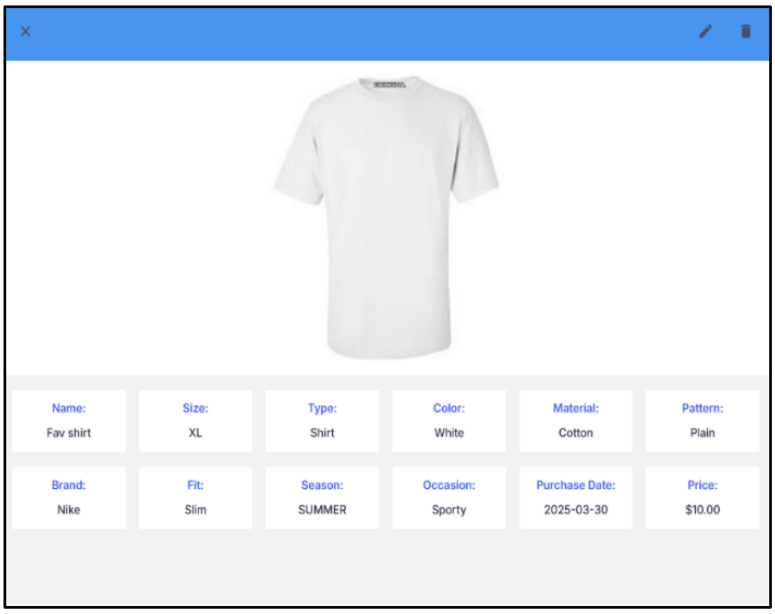
A galéria a webalkalmazás azon része, ahol a felhasználók feltölthetik új, illetve megtekinthetik az összes már feltöltött ruhadarabjaikat. Az utóbbiak kártyaszerű elrendezésben jelennek meg, néhány fontosabb információjukkal.



6. ábra – Az alkalmazás galéria felülete

4.1.4.1. Részletes nézet

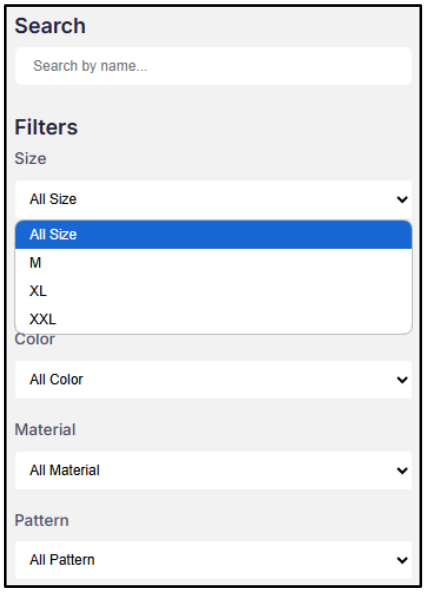
Egy kártyára kattintva megnyitható az adott darab részletes nézete, amely a 7. ábrán is látható, ahol a feltöltéskor megadott minden információ megtalálható, egy nagyobb képpel karöltve. Ezen információk természetesen módosíthatóak, illetve a ruhadarab törlésére is itt van lehetősége a felhasználónak, amennyiben nincs már rá szüksége a továbbiakban. Ezen célok elérésére a jobb felső sarokban található kis ikonok segítenek.



7. ábra - Egy tetszőleges ruhadarabra való kattintás után felugró ablak

4.1.4.2. Szűrés és keresés

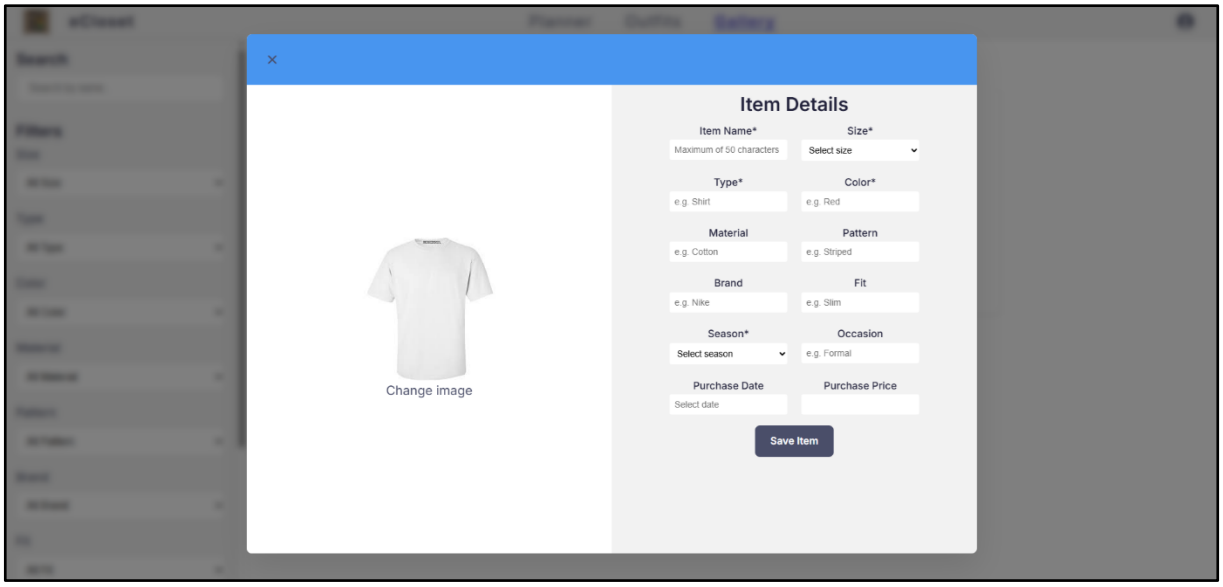
Az oldal egy komplex szűrővel rendelkezik, amely biztosítja, hogy bármely feltöltött elem gyorsan és egyszerűen megtalálható legyen. A feltöltéskor megadott információk szinte mindegyikére lehet szűrni, illetve a keresővel lehetséges a megadott nevekre történő szűkítés is. Fontos, hogy az egyes szűrőknél csak azok a lehetséges opciók jelennek meg, mint szűrési feltétel, amelyek legalább egyszer előfordulnak a feltöltött ruhák adatai között. Tehát, amennyiben csak fehér pólók kerültek feltöltésre, akkor a színnél csak a fehér lesz kiválasztható.



8. ábra - A galéria oldalán található szűrő

4.1.4.3. Új ruhadarab feltöltésének menete

Az „Upload item” gombra kattintás után a felugró ablakon keresztül van lehetőség ruhadarabok feltöltésére. Először a kép kiválasztása szükséges, az adatok megadására csak ezt követően kerül sor.



9. ábra – Az alkalmazás ruhadarab feltöltésére szolgáló felülete

A kiválasztott kép bal oldalon megjelenik egy előnézetben, lehetőséget adva az ellenőrzésre. Amennyiben másik darabot szeretnénk választani, azt a „Change image” feliratra kattintva tehetjük meg, mely törli az előző képet és ezzel egyidejűleg visszarepít a feltöltés felületéhez. Amennyiben elégedettek vagyunk a képpel, a jobb oldalon nyílik meg a kategorizációs lehetőségek tárháza a felhasználó előtt, amely keretein belül az alábbi mezők kitöltése lehetséges:

- Méret (például: XS, S, M, L, XL, XXL, stb.)
- Típus (például: póló, pulóver, nadrág, cipő, sapka, ékszer, egyéb kiegészítő, stb.)
- Szín (például: fekete, fehér, piros, stb.)
- Fazon (például: szűkített, bő, stb.)
- Évszak (ősz, tél, tavasz, nyár)
- Vásárlás dátuma
- Ár
- Anyag (például: pamut, bőr, stb.)
- Minta (például: csíkos, egyszínű, pöttyös, stb.)
- Alkalom (például: hétköznapi, elegáns, sportos, stb.)
- Márka (például: Nike, Adidas, stb.)
- Név

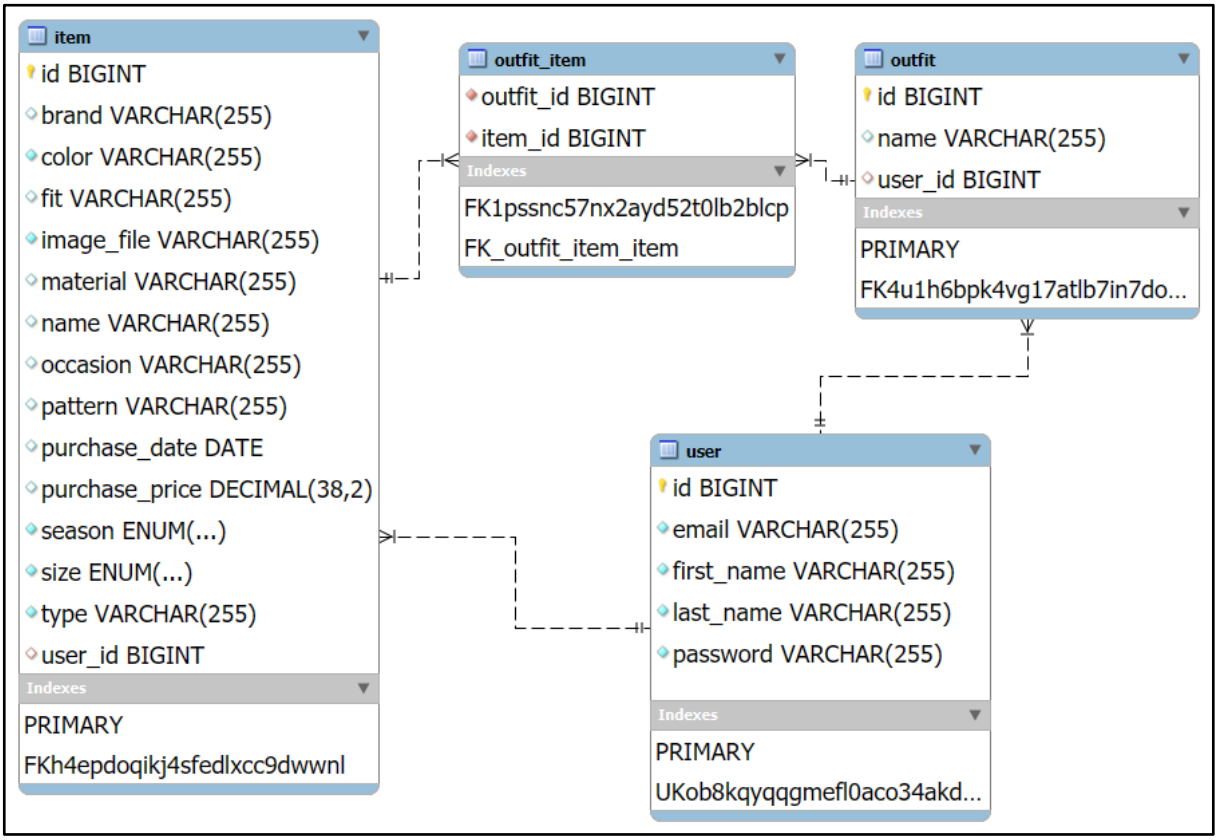
Értelemszerűen nem minden mező kitöltése kötelező, amely mégis az egy apró csillaggal van jelölve az adott mező neve mellett. Az összes kívánt adat megadását követően a „Save Item" gombra kattintva a rendszer automatikusan rögzíti azokat az adatbázisban, illetve elmenti a képet. A sikeres mentést követően a felhasználó automatikusan visszairányításra kerül a „Galéria" oldalra, ahol már látható lesz az újonnan feltöltött ruhadarab.

4.2. Backend

Alkalmazásom backendjének felépítése szigorúan követi az MVC (Model-View-Controller) [33] architekturális tervezési mintát. Ez teszi lehetővé, hogy a rétegek külön-külön kezelhetőek legyenek, javítva a kód fenntarthatóságát, skálázhatóságát, hiszen, ha a későbbiekben módosításra szorulna az alkalmazás, elég csak az adott rétegen elvégezni a szükséges dolgokat.

4.2.1. Az adatbázis

Az alkalmazásom, mint az már korábban is kiderült, a MySQL adatbáziskezelő-rendszert használja, az adatbázis neve pedig `wardrobe_db`. Ezen belül három táblát kezelünk az `item` és `outfit` táblákat összekapcsoló `outfit_item` tábla leszámításával. Az 10. ábrán látható a teljes adatbázis felépítése, a táblák és a közöttük lévő kapcsolattal együtt:



10. ábra - Az alkalmazás adatbázisának ER diagramja

Ezek a táblák az entitás osztályokban megszabott mezők alapján kerülnek generálásra a Hibernate-nek köszönhetően, amely az `@Entity` annotációval ellátott osztályokhoz DDL (Data Definition Language) utasításokat generál. Utolsó lépésként ezek az utasítások végrehajtásra kerülnek, így létrehozva a táblákat az adatbázisban.

```
CREATE TABLE user (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    first_name VARCHAR(255),
    last_name VARCHAR(255),
    password VARCHAR(255) NOT NULL
);
```

11. ábra - példa a Hibernate által generált utasításra

A táblák 3. normálformában [34] vannak, így megelőzve a redundáns adatokat, illetve az `outfit` és `item` közötti kapcsolótábla segít a fölösleges ismétlések elkerülésében. Egyes külső kulcsoknál beállításra került az **`ON DELETE CASCADE`** szabály, így például, ha egy ruhadarab törlésre kerül, az azt felhasználó összeállításokból is törlődni fog.

4.2.1.1. Az item tábla

Ez a tábla felel minden egyes ruhadarabért, illetve azoknak különféle adataiért. Minden egyes adatnak meg van az adott elvárt formátuma, illetve, hogy szükséges-e, hogy értékkel rendelkezzen. Mivel fontos, hogy tudjuk, hogy melyik ruhadarab melyik felhasználóhoz tartozik, így az `item` és `user` tábla egy N:1-es kapcsolatban áll egymással, hiszen egy felhasználóhoz több ruhadarab is tartozhat, viszont egy ruha csak egy felhasználóé lehet. Ezen felül, az `outfit_item` táblán keresztül egy N:M kapcsolat áll fenn az outfit táblával, azaz egy öltözethez több ruhadarab is rendelhető és egy ruha több öltözetben is szerepelhet.

Mezők

- **`id`**: Egyedi azonosító, amely az elsődleges kulcs.
- **`brand, color, material, name, type, fit, occasion, pattern, purchase_date, purchase_price`**: Ruhára vonatkozó tulajdonságok.
- **`image_file`**: A ruhához tartozó kép fájl neve.
- **`season, size`**: Előre definiált `ENUM` értékek (például a négy évszak).
- **`user_id`**: Külső kulcs.

id	brand	color	fit	image_file	material	name	occasion	pattern	purchase_date	purchase_price	season	size	type	user_id
73	Nike	Black	1	71k5Cr+5PIL. AC ...	Denim	Black hat	Sporty	Plain	2025-03-31	25.00	SUMMER	M	Hat	4

12. ábra - Példa egy sorra az item táblából

4.2.1.2. Az outfit tábla

E tábla felelőssége, hogy az összeállítások adatait tárolja. Az `outfit_item` kapcsolótábla segítségével kerül tárolásra az egyes öltözetekhez tartozó ruhadarabok azonosítója, amely lista alapján történik a frontend oldalán az öltözethez tartozó ruhadarabok megjelenítése. A tábla N:1 kapcsolatban áll a `user` táblával, mivel egy felhasználóhoz több outfit is tartozhat.

Mezők

- **`id`**: Egyedi azonosító, amely az elsődleges kulcs.
- **`name`**: Összeállítás neve.
- **`user_id`**: Külső kulcs.

outfit_id	outfit_name	item_id	item_image
29	elso	73	1_71k5Cr+5PIL._AC_...
29	elso	74	2_images_d4911243-...
29	elso	75	3_RV-Basic-Jeans-Sto...
29	elso	76	4_white-plain-color-la...

13. ábra - Példa egy összeállított öltözetre az adatbázisból, amelyben 4 ruhadarab szerepel

4.2.1.3. A user tábla

Ebben a táblában kerül tárolásra minden felhasználói adat. Mivel jogosultságok kezelésére nem láttam szükséget, így arra nem is került sor, ezért nem rendelkeznek a felhasználók jogosultság mezővel. Az `outfit` és `item` táblákkal 1:N kapcsolatban áll, azaz mindkettőből tartozhat több is egy felhasználóhoz, de ez fordítva nem valósulhat meg.

Mezők

- **`id`**: Egyedi azonosító, amely az elsődleges kulcs.
- **`email`**: A felhasználó email címe (UNIQUE megszorítással rendelkezik)
- **`first_name, last_name`**: A felhasználó vezeték- és keresztneve.
- **`password`**: A felhasználó titkosított jelszava.

id	email	first_name	last_name	password
1	johndoe22@example....	John	Doe	$2a$10$OAv.SncFiEdeOHItf59RDuX5De...

14. ábra - Példa egy felhasználóra az adatbázisból

4.3. A Data réteg

Itt történik az adatok perzisztens kezelése, tárolása, itt találhatóak az entitások, amelyek az adatbázis tábláinak leképzését végzik az ORM (Object-Relational-Mapping) segítségével. A JPA és Hibernate segítséget nyújt az adatbázis és Java objektumok közötti hatékony és automatikus leképzésekben. A `JpaRepository`-k pedig egyes adatbázis műveletekhez nyújtanak absztrakciót, automatikusan generált SQL lekérdezésekkel, így minden adatbázis-művelet egy egységes felületet érhető el, elrejtve a konkrét implementációt.

4.3.1. Entitás osztályok

Szerepük, hogy egy leképezhető felépítést biztosítsanak, amely alapján az adatbázis táblái is elkészülnek. Ezekről az osztályokról általánosságban elmondható, hogy az alábbi annotációkat alkalmazzák osztály szinten:

```
@Data
@NoArgsConstructor
@AllArgsConstructor
@ToString
@Entity
@Table(name = "XY")
```

15. ábra - Egy entitás osztályon alkalmazott annotációk

- **`@Table`:** Az adatbázisban az itt megadott nevén szerepel.
- **`@Entity`:** Azt jelzi, hogy az osztály egy JPA entitás.
- **`@Data`:** Elvégzi a `Getter`-ek, `Setter`-ek, `equals()`, `hashCode()` és `toString()` metódusok generálását.
- **`@NoArgsConstructor`, `@AllArgsConstructor`:** Elvégzi a paraméter nélküli, illetve az összes paramétert felhasználó konstruktorok generálását.

4.3.1.1. Gyakran előforduló annotációk

- **`@Id`, `@GeneratedValue(strategy = GenerationType.IDENTITY)`:** Ez a két annotáció együtt dolgozik, ezek alapján generálódik minden tábla elsődleges kulcsa. A `strategy`-vel határozható meg, hogy milyen módon generálódjon ez az azonosító.
- **`@Column`:** Megadja az oszlop nevét az adott táblában.
- **`@Enumerated(EnumType.STRING)`:** Megadja, hogy az adatbázisban sztringként kerüljön tárolásra az adott enum típus.
- **`@ManyToOne`, `@ManyToMany`, `@OneToMany`, `@JoinColumn`:** A táblák összekapcsolásáért és azok kapcsolatának meghatározásáért felelős annotációk.

4.3.1.2. UserEntity

A `UserEntity` osztály különlegessége, hogy nem csak az adatbázisban eltárolandó felhasználói adatokat képviseli, hanem implementálja a Spring Security `UserDetails` interfészét is, amelynek köszönhetően egyszerűbben kezelhető a hitelesítési folyamat, mivel nem szükséges új objektumok létrehozására vagy egyéb konverziókra. Véleményem szerint ez a megközelítés egyszerűbbé és átláthatóbbá teszi a kódot, illetve egységesíti a felhasználókezelést is a különböző rétegek között.

4.3.1.3. ItemEntity

A ruhadarabok osztályánál úgy gondoltam, hogy hasznos lehet egyes mezőknél, kimondottan a méretnél, illetve az évszaknál, az `ENUM`-ok használata, ezzel redukálva a fölösleges és helytelen lehetőségeket, amelyek az adatbázisba kerülhetnek. Az évszak esetében például csak és kizárólag a négy megadott évszakot fogadja el helyes értékként a rendszer, így nem áll fenn annak az esélye, hogy egy elírás miatt több helytelen értéket kell tárolni, de a méretezés esetében is hasonló az eljárás.

4.3.2. Repository

Minden entitás osztályhoz tartozik egy `repository`, könnyítve az azokhoz tartozó adatok kezelését. Ezek mind az `@Repository` annotációval vannak ellátva és a `JpaRepository` interfészt terjesztik ki, amely rengeteg előre definiált metódust biztosít (például `save()`, `findById()`, melyek az entitás mentéséért, illetve azonosító alapján történő keresést biztosítják). Az előre definiált metódusokon felüli a Spring Data JPA lehetőséget biztosít, hogy metódusnevek alapján automatikusan generáljon lekérdezéseket. Az általam használt metódusok is ezt a remek tulajdonságot használják fel.

A `findAllByUserId()` metódusok biztosítják, hogy az adott táblából visszaadják válaszképp az öltözékek vagy ruhadarabok azon listáját, amelyeknél a felhasználó azonosítója megegyezik a megadott azonosítóval.

A `findByEmail()` metódus gondoskodik arról, hogy kiválassza azt a felhasználót a nagy halmazból, amelyiknél megegyezik a megadott email cím a táblában szereplővel.

```
@Repository
public interface ItemRepository extends JpaRepository<ItemEntity, Long> {
    1 usage   Nagy Szabolcs
    List<ItemEntity> findAllByUserId(Long id);
}
@Repository
public interface OutfitRepository extends JpaRepository<OutfitEntity, Long> {
    1 usage   Nagy Szabolcs
    @EntityGraph(attributePaths = {"items"})
    List<OutfitEntity> findAllByUserId(Long id);
}
@Repository
public interface UserRepository extends JpaRepository<UserEntity, Long> {
    3 usages   Nagy Szabolcs
    UserEntity findByEmail(String email);
}
```

16. ábra - Az entitás osztályokhoz tartozó repository-k

4.4. A Service réteg

Központi szerepet játszik az üzleti logika kezelésében, amelyek lehetnek az adatok feldolgozása, azok validációja, `DTO-Entity` konverzió, tranzakciókezelés és egyéb különböző logikai műveletek. Ennek a rétegnek köszönhetően a kontroller osztályokból elvonható a részletek kezelése, ezzel átláthatóbbá téve azok felelősségeit. Emellett a `DTO`-k

alkalmazása segít minimalizálni a különböző rétegek között átvitt adatok terjedelmét, illetve biztosítja, hogy azokból csak és kizárólag a szükségesek kerüljenek továbbításra.

4.4.1. DTO (Data Transfer Object)

A `DTO`-k olyan egyedi objektumok, amelyek feladata a különböző rétegek közötti adatátvitel biztosítása. Legfőbb lényegük, hogy az entitásokat védjék, úgy, hogy egy „tűzfalként" működnek, azaz csak bizonyos adatokat adnak át, csökkentve a belső, érzékeny mezők kiszivárgásának lehetőségét. Ezen felül szerkezetük rendkívül rugalmas, mivel függetlenek az entitások struktúrájától, így könnyen személyre szabhatóak az egyes igényekhez. Mivel ezen osztályok fő szerepköre az adatok átvitele, így alkalmazásomban minden formai megszorítást ezen osztályok mezőin helyeztem el, amelyet különféle annotációk látnak el. Általánosságban elmondható, hogy milyen ilyen annotációnál megadható egy üzenet (`message = "..."`), amely egyfajta hibaüzenetként fogható fel, mivel ezek csak akkor kerülnek felszínre, ha a felhasználó által megadott adat nem felel meg az elvárt megszorításoknak.

4.4.1.1. Felépítésük

Szerkezetükben nagyban megegyeznek az entitások felépítésével. A legfőbb különbségek, hogy az osztályszintű annotációknál itt már nem kerül alkalmazásra az `@Entity` és `@Table`, illetve az entitásosztályokban meghatározott táblák közötti kapcsolatok sem kerülnek újbóli feltüntetésre. Mivel az egyéb mezők megegyeznek az adatbázisnál tábláinál részletesen taglalt felépítéssel, így azok felsorolását az ismétlés elkerülésének érdekében nem ejtem meg újra. Az ábrán található egy részlet, amely a ruhadarabokhoz kapcsolódó `DTO` osztályban valósul meg.

```
@Size(max = 50, message = "The item's name cannot be longer than 50 characters")
private String name;

@NotNull(message = "Sizing is required.")
private Sizing size;

@NotBlank(message = "Type is required.")
private String type;

@NotBlank(message = "Color is required.")
private String color;

@Nullable
private String material;
```

17. ábra - Részlet a ruhadarabokhoz használt DTO osztályból

4.4.1.2. Megszorítások

- **`@Nullable`:** Egy mező helyes lesz, bármilyen felvett érték esetén (akár null is).
- **`@NotNull`:** Egy mező helyes lesz, a null értéken kívül bármilyen egyéb felvett érték esetén (whitespace karakterek is).

- **`@NotBlank`:** Egy mező helyes lesz, ha felvett értékének (whitespace karakterek kivételével) hossza nagyobb, mint nulla és értéke nem null.
    - Az előbbi három annotáció például a ruhadarabok egyes mezőinél kerültek felhasználásra, mely a fentebbi ábrán is látható.
- **`@Email`:** Egy mező helyes lesz, ha értéke követi a megszokott email cím formátumot (például mail@example.com).
    - Ez a megszorítás a bejelentkezésnél és regisztrációnál használatos email cím esetében került alkalmazásra.
- **`@Size`:** Egy mező helyes lesz, ha felvett értékének hossza a megadott tartományon belül helyezkedik el (például maximum 50 karakter).
- **`@PastOrPresent`:** Egy dátum helyes lesz, ha az nem jövőbeli, tehát az aktuális nap vagy attól korábbi. Ez rendkívül fontos, hiszen egy jövőbeli vásárlási dátum nem állja meg a helyét egy ruhadarabnál, mivel többnyire nem tudjuk pontosan, hogy mikor fogjuk megvenni azt.
- **`@JsonFormat`:** Meghatározza a dátum szerializálási formátumát (például „yyyy-MM-dd”, azaz év-hónap-nap).
    - Ez a megszorítás a ruhadarabok vásárlási dátumánál került alkalmazásra.
- **`@Pattern`:** Egy mező helyes lesz, ha az megfelel az ebben az annotációban megadott `regexp` szabálynak.
    - Ez a megszorítás a regisztráció során megadandó jelszó esetén került alkalmazásra.
    - Az alábbi ábrán látható az általam alkalmazott szabály, amely megszabja, hogy a jelszó minimum 8 karakter hosszú kell, hogy legyen és tartalmaznia kell legalább egy kis- és nagybetűs, illetve speciális karaktert, valamint legalább egy számot.

```
@Pattern(regexp = "^(?=.*[A-Z])(?=.*[a-z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{8,}$",
        message = "Password must be at least 8 characters long and contain an uppercase letter," +
                "a lowercase letter, a number, and a special character.")
private String password;
```

18. ábra - A jelszó elvárt formátumát meghatározó reguláris kifejezés

4.4.2. Service osztályok

Ezen összese használja az `@Service` osztályszintű annotációt, amely arra utasítják a keretrendszert, hogy az adott osztályt automatikusan vegye fel a Spring kontextusba, mint egy

`bean`. Ez nyitja meg a lehetőséget arra, hogy az osztály példányosítható és injektálható legyen (például `@Autowired` a Controller osztályokban), illetve fontos szerepe van a komponensek életciklusának kezelésében és a függőségek automatikus bekötésében.

A szolgáltatásokat megszokott interfész és implementáció formában elkülöníteni, amely elősegíti a laza csatoltságot, azaz a komponensek kevésbé függenek egymástól közvetlenül. Ennek köszönhetően sokkal egyszerűbben módosítható vagy tesztelhető egy-egy implementáció. Mindezeken felül hatalmas előnyt jelent a kód karbantarthatóságában és skálázhatóságában, mivel, ha például az üzleti követelmények változnak elegendő csupán az implementáció változtatása.

4.4.2.1. Metódusok általános logikája

Ezen osztályokban található metódusok feladata az, hogy a Controller rétegből érkező adatokat feldolgozzák, majd egy megfelelő válasszal szolgáljanak, amennyiben az szükséges. E folyamatnak három fő lépése van, bár nem minden esetben kerül mindhárom végrehajtásra, hiszen ez az adott metódus által elvégzendő feladat szerint dől el.

A három fő lépés

1. **DTO -> Entity konverzió:**
   - Mivel a kliens részéről érkező adatok rendszerint egy `DTO` formájában érkeznek meg a backendhez, így minden esetben szükséges azok `Entity`-vé alakítása, mivel az adatok perzisztens tárolása csak ilyen objektumokon keresztül történhet meg. A konverzióhoz a ModelMapper könyvtárat használom (illetve néhány esetben manuálisan végzem azt el), mivel ez automatikusan képes azonos nevű mezőket átkonvertálni egyik típusból a másikba, illetve feladatát kifejezetten hatékonyan végzi.
2. **Műveletek végrehajtása:**
   - A konverziót követően szabadon kezelhetőek az `Entity` objektumok, legyen szó azok elmentéséről az adatbázisba vagy akár az onnan való törlésről. Ezek a műveletek a `Repository`-k használatával történnek meg, amelyek tényleges műveleteket hajtanak végre az adatbázison.
3. **Entity -> DTO konverzió:**
   - Amennyiben a műveletek végrehajtásából származó eredményt vissza kell küldeni a kliens részére (például az elmentett ruhadarabok adatait), az `Entity` objektumot

DTO-vá szükséges alakítani. Ez teszi lehetővé, hogy csak a szükséges mezők kerüljenek a kliens felé küldött válaszba.

Általánosságban elmondható még, hogy a legtöbb metódusnál alkalmazásra került az alábbi ábrán látható kódrész:

```
Long currentUserId = ((UserEntity) SecurityContextHolder.getContext()
        .getAuthentication().getPrincipal()).getId();
```

19. ábra - Kódrészlet, amely segítségével történik meg az éppen bejelentkezett felhasználó azonosítójának lekérdezése

Ennek a bonyolultnak tűnő sornak a feladata, hogy a jelenleg bejelentkezett felhasználó azonosítóját tárolja, amely alapján különféle biztonsági lépések kerülnek végrehajtásra.

4.4.2.2. AuthService

Az felhasználói fiókok kezeléséért, illetve hitelesítéséért felelős logikát valósítja meg, azaz feladata a regisztráció és bejelentkezés kezelése. Mindez természetesen különféle biztonsági eljárásokkal karöltve történik.

A regisztráció során például a konverziót követően a `PasswordEncoder.encode()` metódus segítségével titkosításra kerül a jelszó, s az adatbázisba már ebben a formátumban kerül elmentésre az entitás. A `PasswordEncoder` egy interfész, amely a Spring Security által kerül biztosításra.

```
@Override
public void registration(RegistrationDto dto) {
    UserEntity entity = modelMapper.map(dto, UserEntity.class);
    entity.setPassword(passwordEncoder.encode(entity.getPassword()));

    userRepository.save(entity);
@Override
public String login(LoginDto dto) {
    var user = userRepository.findByEmail(dto.getEmail());

    if (user != null && passwordEncoder.matches(dto.getPassword(), user.getPassword())) {
        return jwtAuthService.generateToken(user);
    } else {
        throw new BadCredentialsException("Invalid username or password");
    }
}
```

21. ábra - Az AuthService által használt bejelentkezésért felelős metódus

A bejelentkezés során először megnézzük, hogy az email cím alapján létezik-e egyáltalán a felhasználó, illetve a `PasswordEncoder.matches()` metódusával megvizsgáljuk, hogy a DTO-ban szereplő nyers jelszó megegyezik-e az adatbázisban tárolt titkosítottal. Amennyiben

mindkét feltétel egyszerre teljesül, a `JwtAuthService` osztály segítségével egy token-t generálunk, amelyet vissza is térítünk egy `String` formátumában.

4.4.2.3. JwtAuthService

A JWT alapú hitelesítés mechanizmusának a megvalósításáért felelős osztály. Feladatai, hogy a felhasználó bejelentkezése során egy egyedi token-t generáljon a fiókjához kapcsolódó információk alapján, illetve a későbbi védett végpontra indított HTTP kérések során ellenőrizze azok érvényességét.

A token generálásáért a `generateToken()` metódus felelős. A generált token tartalmazza a felhasználó email címét, a kiállítási időt és a lejárati dátumot (a generáláskori időpont + 15 perc). A token aláírásához egy titkos kulcs kerül használatra, amelyet a `getKey()` metódus biztosít.

```
@Override
public String generateToken(UserDetails userDetails) {
    return Jwts.builder()
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis()+900000))
            .signWith(getKey())
            .compact();
}
```

22. ábra - A JwtAuthService által biztositott token-t generáló metódus

A token validálásáért a `validateToken()` metódus felelős. Egy token akkor lesz érvényes, ha nem járt le, illetve a benne tárolt email cím megegyezik a validálást kérő felhasználóéval.

```
@Override
public boolean validateToken(String token, UserDetails userDetails) {
    Date date = extractClaim(token, Claims::getExpiration);
    boolean isExpired = date.before(new Date());
    final String email = extractClaim(token, Claims::getSubject);
    return (email.equals(userDetails.getUsername()) && !isExpired);
}
```

23. ábra - A JwtAuthSerive által biztositott token validálásért felelős metódus

Az osztályban található még továbbá egy `extractEmail()` és egy `extractClaims()` metódus is. Ez a két metódus arra szolgál, hogy a token-ből kinyerje az e-mail címet (vagy mást). A `extractEmail()` metódus meghívja az `extractClaim()` nevű segédfüggvényt, amely a token tartalmát (payload-ját) dolgozza fel. Először ellenőrzi a token aláírását egy kulccsal, majd kinyeri belőle a szükséges információt. Jelen esetben ez az alany mező, amely az e-mail címet tartalmazza. Ez a megoldás biztonságosan és rugalmasan teszi lehetővé különböző adatok kiolvasását a token-ből.

4.4.2.4. UserService

A Spring Security hitelesítési folyamatának egy kulcslépéséért felelős osztály, amely nem mást, mint a felhasználói adatok betöltése. A Spring Security rendszere a `UserDetailsService` interfészt használja, hogy ellenőrizze a hitelességet a bejelentkezni kívánó felhasználók adatai alapján. Ennek a `loadUserByUsername()` metódusa hívódik meg minden esetben, amikor az email cím alapján szeretnénk hozzáférni a felhasználó minden adatához. Ha az adatbázisban szerepel a megadott email cím, a metódus visszatér a `UserDetails` interfészt implementáló felhasználói entitásosztállyal, amelyben minden adat megtalálható. Ellenkező esetben `UsernameNotFoundException` dobódik.

```
@Override
public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
    UserEntity user = userRepository.findByEmail(email);
    return user;
}
```

24. ábra - A UserService-ben használt loadUserByUsername metódus

4.4.2.5. ItemService

Az `ItemService` osztály végzi az alkalmazás egyik legfontosabb feladatát, amely nem mást, mint a ruhadarabok kezelése.

Legfontosabb feladatok

- **Ruhadarab mentése:** A kliens felől érkező `DTO`-k, amelyek egy-egy ruhadarab leírását tartalmazzák először itt is átalakításra kerülnek egy `Entity` objektummá, amelynél először beállításra kerül a feltöltést végző felhasználó azonosítója, majd ezt követően megtörténik az adatok mentése az adatbázisba.
- **Ruhadarab törlése:** A ruhadarabok törlése az azonosítójuk alapján történik, mellyel a felhasználói felületen alap esetben nem lehet probléma, viszont például a Postman használatával képesek lennénk bármilyen azonosítójú darabot törölni. Ennek elkerülése érdekében született meg a feltétel, miszerint egy adott ruhadarab csak akkor törölhető, ha az ahhoz elmentett azonosító megegyezik a törlési kérést indító felhasználó azonosítójával (valamint természetesen létezik a törölni kívánt azonosítójú ruhadarab az adatbázisban).
- **Ruhadarabok listázása:** A bejelentkezett felhasználó azonosítója alapján visszaadja az összes adatbázisba elmentett ruhadarabot.

Képek kezelése

Ugyan léteznek megoldások arra, hogy a képek az adatbázisban kerüljenek eltárolásra (például BLOB – Binary Large Object), ez a megközelítés nem ajánlott, mivel a fájlméretek miatt könnyen teljesítménybeli problémákba, illetve rengeteg egyéb negatív hatásba ütközhetünk. A szolgáltatásban a képek feltöltéséért a `saveImage()` metódus felel, mely keretein belül lokálisan tárolódik minden kép egy erre kialakított mappában (az „`C:\Users\user_name\AppData\Local\Virtual Wardrobe\Uploaded Images\user_id\...`" elérési útvonalon), az adatbázisban pedig csak a megfelelő fájlnevek kerülnek tárolásra.

Amennyiben ez a célkönyvtár még nem létezik, az létrehozásra kerül, ezt követően pedig a fájlnév beállítása következik. Alaphelyzetben a feltöltése kerülő fájlok felülírnák egymást, ha a nevük megegyezik, ennek elkerülése érdekében vezettem be az alábbi módosításokat:

- Az eredeti fájlnévből kinyerésre kerül maga a név és kiterjesztés.
- A névhez hozzáfűzésre kerül egy automatikusan növekvő sorszám, illetve egy véletlenszerű `UUID` [35] (Universal Unique Identifier, ami egy 128 bites egyedi azonosítót generál), majd a végére visszacsatolásra kerül a kiterjesztés.

```
String originalName = file.getOriginalFilename();
String baseName = originalName.substring(0, originalName.lastIndexOf( ch: '.'));
String extension = originalName.substring(originalName.lastIndexOf( ch: '.'));

String newFilename = counter.getAndIncrement() + "_" + baseName + "_" + UUID.randomUUID() + extension;
```

25. ábra - Az ItemService saveImage() metódusa által használt névgenerálás kódrészlete

Miután az egyedi név beállításra került, a `Files.copy()` metódus segítségével elmentésre kerül a kép a meghatározott útvonalra és a metódus visszatér az egyedi fájlnévvel, hogy az elmentésre kerülhessen az adatbázisba. Az említett metódus a Java egy beépített, kifejezetten fájlkezelésre szánt osztályából származik, melynek neve a `Files`.

A képek betöltése a `loadItemImage()` metódussal történik, amely a mentéskor meghatározott fájlnév alapján megkeresi a megfelelő képet és visszatér azzal.

4.4.2.6. OutfitService

Akár csak az `Entity` és `DTO` osztályoknál, itt sem sok eltérés van az `OutfitService` és ItemService osztályok között. A CRUD műveletek végrehajtása lényegében egy és ugyan az, csak nyilván az öltözékek paramétereihez igazítva. Képek kezelésére itt sincs szükség, hiszen ez a funkció az `Item`-ek osztályában került implementálásra. A szemnek elsőre feltűnő különbség az, hogy az öltözetek elmentéséért felelős `saveOutfit()` metódus több féltételen halad át, mielőtt a tényleges mentés megtörténhet. Először azt vizsgáljuk, hogy a felhasználó által elküldött `DTO`-ban szereplő és az adatbázisban szereplő `Item`-ek mérete megegyezik-e. Ez azért szükséges, mert ha bármilyen módon a kérésbe (például Postman [36] használatával) olyan azonosítójú ruhadarab kerül, amely nem is létezik, fontos, hogy az öltözék ne legyen elmenthető. A második lépés az `ItemService`-nél is említett probléma orvosolására szolgál, miszerint a bejelentkezett felhasználó azonosítója meg kell, hogy egyezzen az elmentett ruhadaraboknál tárolt felhasználói azonosítóval, azaz a felhasználó csak az általa „birtokolt” ruhákat kezelhesse.

```
if (itemEntityList.size() != dto.getItems().size()) {
    throw new RuntimeException("One or more item ID is not valid.");
}

itemEntityList.forEach(item -> {
    if (!item.getUser().getId().equals(currentUser.getId())) {
        throw new RuntimeException("Item " + item.getId() +
                " does not belong to the current user.");

    }
});
```

26. ábra - Kódrészlet a feltételekről, amelyeknek meg kell felelnie egy öltözetnek a mentése során

A `getAllOutfits()` metódus abban különleges, hogy a felhasználó által birtokolt összes öltözetet egy olyan formátumban adja vissza, ahol az öltözet neve, azonosítója és a ruhák azonosítói mellett szerepelnek az azokhoz kapcsolódó képek fájlneve is, ezzel megkönnyítve a vizuális megjelenítéshez szükséges folyamatot és csökkentve a lépések számát.

```
return outfits.stream().map(outfit -> {
    List<ItemEntity> items = outfit.getItems();

    return new OutfitDto(
            outfit.getId(),
            outfit.getName(),
            items.stream().map(ItemEntity::getId).toList(),
            items.stream().map(ItemEntity::getImageFile).toList()
    );
}).toList();
```

27. ábra - A getAllOutfits() metódusban használt visszatérési értéket meghatározó kódrészlet

4.5. A Controller réteg

Az itt található osztályok feladata a HTTP kérések, válaszok kezelése. Ezek felelősek azért, hogy a felhasználótól érkező kéréseket fogadják, majd a hozzájuk megfelelő üzleti logikát meghívják. Ennek végrehajtását követően, amennyiben releváns, a megfelelő válasz visszaküldésre kerül a felhasználó számára (például 200-as állapotkód, mely a kérés sikerességét jelzi az ügyfélnek).

4.5.1. Gyakran előforduló annotációk

- **`@RestController`:** Ez az annotáció jelzi, hogy az osztály RESTful webszolgáltatásokat valósít meg. Ilyenkor automatikusan alkalmazásra kerülnek a `@Controller` és `@ResponseBody` annotációk, amelyeknek köszönhetően a metódusok visszatérési értékei JSON formátumban kerülnek visszaadásra.
- **`@RequestMapping`:** Ez az annotáció osztály szinten meghatározza az összes metódusnak az alapértelmezett `URL`-jét, azaz minden metódus ezen az útvonalon belül lesz elérhető (például `/auth/login`).
- **`@Autowired`:** Ez az annotáció teszi lehetővé a függőséginjekciót, ezzel egyszerűbbé téve a komponensek kapcsolatainak kezelését. Az `AuthService` és `JwtAuthService` is ennek használatával kerül injektálásra, így egyszerűen használható az általuk biztosított üzleti logika (például regisztrálás, belépés).

4.5.2. AuthController

Az itt található végpontok felelősek a felhasználók ki- és belépésének kezeléséért, illetve a státuszuk ellenőrzéséért. Alapvető útvonaluk a `"/auth/…"`

4.5.2.1. Regisztráció, belépés, kijelentkezés

Mindhárom metódus a `@PostMapping("/…")` annotációval került ellátásra, mivel minden esetben egy `POST` kérést kezelünk. A belépés és regisztráció metódusainak paramétereinél található `@Valid` és `@RequestBody` annotációk a kérés törzsében érkező `JSON`-t a megfelelő `DTO` objektummá alakítják, illetve ellenőrzik a mezők helyességét a `DTO`-ban megadott feltételek alapján (például `@NotBlank`, azaz nem üres az adott mező). Amennyiben a kérés sikeres volt, a metódusok 200-as vagy 201-es státuszkóddal térnek vissza, illetve egy megfelelő üzenettel. Sikertelenség esetén a 400-as kód és egy hibaüzenet az, ami a visszatérési érték lesz. Ezek alapján könnyen kezelhetőek a frontenden mutatott visszajelzések, így biztosítva a felhasználó folytonos tájékoztatását.

A bejelentkezés metódusa annyiban különbözik, hogy itt beállításra kerül egy `Cookie`, amely a `JWT` által generált token-t fogja tartalmazni.

```
@PostMapping("/login")
public ResponseEntity<String> login(@Valid @RequestBody LoginDto dto, @CookieValue(name = "jwt", required = false) String cookie) {
    if (cookie != null) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body("Already logged in.");
    }
    try {
        String token = authService.login(dto);

        ResponseCookie jwtCookie = ResponseCookie.from( name: "jwt", token)
                .httpOnly(true)
                .path("/")
                .maxAge( maxAgeSeconds: 15*60)
                .sameSite("Lax")
                .build();

        return ResponseEntity.ok()
                .header(HttpHeaders.SET_COOKIE, jwtCookie.toString())
                .body("Successful login.");
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage());
    }
}
```

28. ábra - Az bejelentkezéshez szükséges végpont kódja az AuthController osztályban

Bejelentkezés előtt fontos megvizsgálnunk, hogy nincs-e már aktív `sütije` a felhasználónak „`jwt`” névvel, mivel ebben az esetben fölösleges lenne újat generálnunk. Ennek köszönhetően addig nem indítható új kérés, amíg nem léptünk ki, azaz az előző `süti` nem került törlésre. Amennyiben nem rendelkezünk még `Cookie`-val, a bejelentkezés során generált token elhelyezésre kerül egy maximum 15 percig érvényes `sütiben`, amely ezt követően visszaküldésre kerül egy 200-as státuszkód mellett.

A kijelentkezés során ennek pontosan az ellentétje történik, ugyanis, ha a felhasználó nem rendelkezik „`jwt`” nevű `sütivel`, akkor egy hibaüzenetet kap, miszerint nincs is belépve. Amennyiben még is rendelkezne egyel, annak érvényes élettartama (`maxAge`) nullára kerül beállításra, így az azonnal törlődik.

4.5.2.2. Hitelesítés állapota

A hitelesítés állapotát a `getAuthStatus()` metódussal ellenőrzöm, amelyben szintén a Cookie létezése az irányadó. Ez azért is szükséges, hogy az oldal frissítésekor vagy elhagyásakor a felhasználónak ne kelljen újra belépnie, mivel állapotmentes munkamenetet használok. Amennyiben létezik a megfelelő `süti`, 200-as kóddal jelzésre kerül, hogy a felhasználó be van lépve.

4.5.3. ItemController

Az itt található végpontok felelősek a ruhadarabok kezeléséért. Az osztály a teljes `CRUD` (`Create, Read, Update, Delete`) műveletsor elvégzéséhez szükséges metódusokat megvalósítja, így a ruháink teljeskörű kezelése adott. A végpontok alapértelmezett elérési útvonala az "`/api/…`".

4.5.3.1. @PostMapping("/items")

Ez a végpont felel a ruhadarabok adatainak mentésére szolgáló `HTTP POST` kérésekért. Az `@Valid` annotáció itt is az adatok helyességét vizsgálja. Az `itemService` által nyújtott `saveItem()` metódussal kerül mentésre az adott ruhadarab adatai az adatbázisba, mely sikeressége esetén 201-es kódú, ellenkezőleg pedig 400-as kódú választ adunk vissza, mindkét esetben informatív üzenettel.

4.5.3.2. @GetMapping("/items")

Ez a végpont kezeli a `HTTP GET` kéréseket az itemService `getAllItems()` metódusának meghívásával. Válaszként egy listával tér vissza, amelyben az adott felhasználó által feltöltött ruhadarabokat tartalmazza.

4.5.3.3. @DeleteMapping("/items/{id}")

Ez a végpont végzi a `HTTP DELETE` kéréseket az `itemService deleteItemById()` metódusának meghívásával. A paraméterként megadott azonosító alapján az adatbázisból törlésre kerül az azzal megegyező azonosítójú ruhadarab.

4.5.3.4. Képek kezelése

Az ábrán látható metódusok kezelik a képek elmentéséért és lekérdezéséért felelős végpontokat. Az `itemService` által nyújtott `saveImage()` metódus az előre definiált elérési útra menti el az adott képet egy egyedi névvel. A `loadItemImage()` metódus felel azért, hogy a felhasználó által az adatbázisba elmentett ruhákhoz kapcsolódó fájlneveket megkeresse az adott útvonalon, majd ezt válaszképp visszaadja.

```
@PostMapping("/items/image")
public ResponseEntity<?> uploadItemImage(@RequestParam("file") MultipartFile file) {
    try {
        String fileName = itemService.saveImage(file);
        return ResponseEntity.ok().body(Map.of( k1: "filename", fileName));
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(e.getMessage());
    }
}

Nagy Szabolcs
@GetMapping("items/image/{imageName}")
public ResponseEntity<Resource> getItemImage(@PathVariable String imageName) throws IOException {
    return ResponseEntity.ok().body(itemService.loadItemImage(imageName));
}
```

29. ábra - A ruhadarabok képének elmentésére és lekérdezésére szolgáló végpontok kódja az ItemController osztályból

4.5.4. OutfitController

Az itt található végpontok kezelik az öltözetek teljes `CRUD (Create, Read, Update, Delete)` műveletsorát, így azoknak teljeskörű kezelése adott. Ezek alapértelmezett elérési útvonala az `"/api/..."`. Az itt használt metódusok nagy mértékben megegyeznek a ruhadarabok kezeléséért felelős kontroller osztályban található metódusokkal, a fő különbségek

az URL, amelyen elérhetőek a végpontok, a használt szolgáltatás neve, illetve az, hogy itt már nem történik képkezelés.

4.6. Biztonság, hitelesítés és egyéb konfigurációk

A projektemben alkalmazott biztonsági megoldások középpontjában a JWT alapú hitelesítés és a Spring Security áll. Az alábbiakban ezek megvalósításának körülményeit és a hozzájuk tartozó konfigurációkat részletezem.

4.6.1. JWT (JSON Web Token) alapú hitelesítés

A JWT egy elterjedt megoldás a felhasználók hitelesítésére és az adatok kezelésére webalkalmazásokban. Alapja egy titkosított token, amely JSON formátumban tárolja a felhasználó adatait. Három részből áll, amelyek pontokkal vannak elválasztva egymástól (xxxx.yyyy.zzzz), amelyek sorra a következő elemeket tartalmazzák:

- Header/Fejléc: A token típusa és a titkosítási algoritmus.
- Payload/Tartalom: A felhasználóhoz kapcsolódó információk és egyéb metaadatok.
- Signature/Aláírás: A token hitelességének ellenőrzésére szolgáló kód.

A bejelentkezés során egy ilyen token kerül generálásra a JwtAuthService osztály által, illetve elmentésre egy Cookie-ban.

A szerver felé intézett kérések egy filteren mennek keresztül, amelyet jelent esetben a JwtAuthFilterConfiguration osztályban meghatározott folyamat végez. Ha a HTTP kérés tartalmaz egy token-t, az validálásra kerül a filteren áthaladás során és beállítja a megfelelő hitelesítést a felhasználó számára. Mivel a konfigurációs osztály kiterjeszti a Spring Security által biztosított OncePerRequestFilter-t, így ez a validáció minden kérés esetén le fog futni.

4.6.1.1. Fontos lépések

- **JWT kinyerése:** A filter a Cookie header-ből nyeri ki a token-t, amennyiben a süti nem tartalmaz „jwt=” részletet, a kérelem elutasításra kerül.

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) throws ServletException, IOException {
    final String authHeader = request.getHeader( s: "Cookie");
    final String jwt;
    final String email;

    if (StringUtils.isEmpty(authHeader) || !StringUtils.startsWithIgnoreCase(authHeader, prefix: "jwt=")) {
        filterChain.doFilter(request, response);
        return;
    }
```

30. ábra - Kódrészlet, amely a token jelenlétet ellenőrzi a Cookie-ban

- **Email kinyerése:** A token-ből a `jwtAuthService.extractEmail()` metódussal kinyerésre kerül az email cím, amely, ha helyes, nem üres és nincs még hitelesítve, akkor az `authenticateUser()` metódus ellenőrzi a token érvényességét a `jwtAuthSerive.validateToken()` metódussal.

```
jwt = authHeader.substring( beginIndex: 4);
email = jwtAuthService.extractEmail(jwt);

if (StringUtils.isNotEmpty(email) &&
        SecurityContextHolder.getContext().getAuthentication() == null) {
    authenticateUser(request, jwt, email);
}

filterChain.doFilter(request, response);
```

31. ábra - A token-ből történő email cím kinyeréséért felelős kódrészlet

- **Hitelesítés beállítása:** Amennyiben a token érvényes, a `SecurityContext` frissítésre kerül a `setSecurityContext()` metódussal.

```
private void setSecurityContext(HttpServletRequest request, UserDetails userDetails) {
    SecurityContext context = SecurityContextHolder.createEmptyContext();

    UsernamePasswordAuthenticationToken authenticationToken = new UsernamePasswordAuthenticationToken(
            userDetails, credentials: null, Collections.emptyList());

    authenticationToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
    context.setAuthentication(authenticationToken);
    SecurityContextHolder.setContext(context);
}
```

32. ábra - A setSecurityContext() metódus

4.6.2. Spring Security

A Spring Security egy kiváló keretrendszer az alkalmazások biztonságossá tételére. Alapvetően a működése a `JWT`-hez hasonlóan egy filterekből álló lépéssorozatra épül (filter chain), ahol minden `HTTP` kérés egy sor szűrőn halad át, mielőtt bármi fontos logikához elérne az adott kérés. A szűrők felelősek például a hitelesítésért (`JWT`), a `CORS` (`Cross-Origin Resource Sharing`) kezeléséért, a `CSRF` (`Cross-Site Request Forgery`) védelemért és a jogosultságok ellenőrzéséért is, bár az utóbbi kettőre alkalmazásomban nem kerül sor. A konfigurálásához egy osztályt szükséges létrehoznunk, amelyet a `@Configuration` és `@EnableWebSecurity` annotációkkal kell ellátnunk. Az előbbi azt jelzi, hogy az osztály egy Spring konfigurációs osztályként kezelendő, az utóbbi pedig aktiválja a Spring Security-t és felülírja annak alapértelmezett beállításait.

4.6.2.1. A konfigurációs fájl

Fő eleme a `securityFilterChain()` metódus, amelyben személyre szabhatjuk a filtereket. A konkrét megszorítások a 33. ábrán láthatóak.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity) throws Exception {
    httpSecurity.csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(req -> req
                    .requestMatchers("/auth/**").permitAll()
                    .anyRequest().authenticated())
            .sessionManagement(sessionManager -> sessionManager.sessionCreationPolicy(STATELESS))
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .cors(cors -> cors.configurationSource(corsConfigurationSource()));
    return httpSecurity.build();
}
```

33. ábra - A Spring Security konfigurációs fájlában használt filter chain

- **`.authorizeHttpRequests`:** Ezen belül adhatjuk meg, hogy mely útvonalakhoz szükséges hitelesítés, illetve, hogy melyekhez nem. A `.permitAll()` jelzi azt, hogy az adott útvonalon minden kérés engedélyezett, függetlenül a hitelesítés státuszától. Az `.anyRequest().authenticated()` szabályozza azt, hogy minden, az előzőleg megadott útvonalakon kívüli kérésekhez szükséges, hogy a felhasználó hitelesített legyen.
- **`.sessionManagement()`:** Mivel állapotmentes munkamenetet használ az alkalmazás, így minden kérésnél szükséges a hitelesítő token jelenléte.
- **`.addFilterBefore()`:** A hitelesítéshez használandó egyedi filtert adható meg itt, amely nálam jelen esetben az előbbiekben tárgyal `JwtAuthFilter`.
- **`.cors()`:** A Cross-Origin Resource Sharing (eredetközi erőforrás-megosztás) egyedi beállításaihoz használandó konfigurációnak a forrása adható meg itt.

4.6.2.2. CORS (Cross-Origin Resource Sharing)

Ezen keresztül engedélyezhető, hogy a frontend meghívhassa a kontroller osztályokban definiált végpontokat.

```
corsConfiguration.addAllowedOrigin("http://localhost:3000");
corsConfiguration.addAllowedMethod("*");
corsConfiguration.setAllowCredentials(true);
corsConfiguration.addAllowedHeader("*");
```

34. ábra - A CORS konfigurációjáért felelős kódrészlet

- `.addAllowedOrigin()`: Az `URL`, amelyet a frontend használ.
- `.addAllowedMethod()`: Az engedélyezett `HTTP` metódusokat szabja meg.
- `.addAllowedHeader()`: Az engedélyezett `Header`-eket szabja meg.
- `.setAllowCredentials()`: „`true`” értékkel engedélyezi, hogy a frontendünk például `Cookie`-kat is küldhessen.

5. Lehetőségek az alkalmazás továbbfejlesztésére

Az alkalmazás továbbfejlesztésére számos opciót tartok számon, amelyek mind javíthatják a felhasználói élményt, növelhetik az interaktivitást és még hasznosabbá tehetik a felületet.

5.1. Automatikus öltözetajánlás

Napjain egyik legelterjedtebb fejlesztési iránya az AI alapú megoldások használata. Webalkalmazásomban ez megvalósításra kerülhetne egy AI alapú öltözködési ajánlórendszer keretein belül, amely a felhasználó korábbi választásait, az időjárási előjelzéseket és a személyes preferenciákat figyelembe véve automatikusan javasolhatna öltözeteket. Egy ilyen rendszerrel továbbá gyorsítható lenne az öltözködés folyamata.

5.2. Képfelismerés a ruhadarabok azonosítására

Mivel jelenleg egy darabhoz tartozó minden adat megadása manuálisan történik, így ez a folyamat viszonylag időigényes. Az AI alkalmazásával ez a folyamat automatizálható lehetne, hiszen elegendő lenne a kép feltöltése, a különféle adatok pedig automatikusan feltöltésre kerülnének. Ez jelentősen felgyorsítaná az adatbevitel folyamatát.

5.3. Naptár integráció

A felhasználók számára biztosítható lenne egy személy naptár, amely segítségével könnyebben megtervezhető lenne az elkövetkezendő napokon viselni kívánt öltözet. E funkció keretein belül is számos lehetőség rejlik, hiszen egy napnál például megjelölhető lenne az időtartam, amelyen keresztül rajtunk lesznek az adott ruhák, vagy akár megjelölésre kerülhetnének fontosabb események is. Ezen felül akár egy értesítési rendszer is bevezethető lenne, amely email-es emlékeztetők keretein belül tájékoztatja különféle dolgokról a felhasználót.

5.4. Időjárás előrejelzés

Mivel a viselt ruhákat nagyban befolyásolja az aznapi időjárás alakulása, így ezt a funkciót tartom a legfontosabbnak. Ugyan különböző alkalmazások, oldalak használatával megtekinthető az időjárás, viszont sokkal hatékonyabbá válna a webalkalmazásom, ha ehhez meglenne az azon belül kialakított egyedi felület.

5.5. Indexelés az adatbázisban

Az alkalmazás adatbázisa kulcsfontosságú szerepet játszik a rendszer gyors működésében, különösen, ha nagyobb adathalmazokról van szó. Az adatbázisban történő indexelés segíthetne gyorsítani a gyakran használt lekérdezéseket (például a felhasználó ruhadarabjainak keresése esetében).

6. Összefoglalás

Az alkalmazás fejlesztése során sok olyan egyedi problémába ütköztem, melyekkel eddig még nem találkoztam, így rengeteg új tapasztalattal bővült az ismeretem, bár ez főképp a frontend területére értendő. Ugyan korábban már minden technológiával és eszközzel találkoztam, amelyek projektemben felhasználásra kerültek, a hozzájuk kapcsolódó tudásom mélysége mindegyiknél értérő. A frontend területén használt React-tal dolgoztam a legkevesebbet, így ennek gyakorlati alkalmazásában mélyebb szintű tudás megszerzésére mindeddig nem volt alkalmam. Az internetes keresések és hivatalos dokumentációk böngészése során számos best practice-ekre bukkantam, amelyek úgy gondolom, hogy hosszú távon is segíteni fogják a munkámat. Szakdolgozatomat megelőzőleg csak néhány nagyon egyszerű weboldalt alkottam a React használatával, viszont ezek során rengeteg olyan funkcióval nem sikerült találkoznom, amelyeket jelen projektemben már aktívan használok. A legnagyobb küzdelem a komponensek helyes strukturálását jelentette, hiszen erre nincs kimondott szabvány, amely követendő, viszont úgy érzem, hogy a projektemben sikerült egy olyan struktúrát összeállítani, amelyet széleskörben tudok majd alkalmazni a későbbiek során. Megtanultam továbbá a CSS modulok használatát, melyek nagyobb kódbázisnál fontos szerepet játszanak az olvashatóságban. Mindezek mellett pedig számos elkerülendő hibára is felfigyeltem, amelyek tudatlan fejjel nem voltak egyértelműek, így rendkívül örülök, hogy az elkövetkezendőkben már ezek kijavításával nem kellesz értékes órákat eltöltenem.

A frontend oldalán rengeteg időt töltöttem a megfelelő oldalszerkezetek kialakításával, illetve a stílusokkal, mivel az UI tervezés sosem volt az erősségem. Nehéz volt a felhasználói élmény javítása és az egyszerűség hangsúlyban tartása, de úgy gondolom, hogy sikerült megtalálnom az aranyközéputat.

A backend fejlesztése viszonylag gördülékenyen ment, mivel ezen a területen rendelkeztem a legtöbb tapasztalattal és ismerettel. Mind az adatbázis megtervezése és felépítése, az entitások, DTO-k és végpontok elkészítése, illetve a Spring Security integrálása egy olyan folyamat volt, ahol nagyobb hibák elkövetése nélkül, viszonylag gyorsan haladtam. Korábbi tapasztalataim alapján döntöttem újra a ModelMapper használata mellett, mivel már korábbi projektjeimben is rendkívül hasznosnak bizonyosult, illetve használatát rendkívül könnyűnek tartom. A problémásabb részek a JWT-alapú autentikációs rendszer kiépítése, illetve a Service osztályok megalkotása volt. A JWT-vel keveset dolgoztam, így rengeteg helyen elakadtam a fejlesztés során, illetve a kapcsolódó dokumentációkat sem tudtam minden esetben tökéletesen

értelmezni. A Service osztályok logikájának implementálása volt a legidőigényesebb feladat, mivel fontos, hogy minden adat megfelelően kerüljön kezelésre, viszont többnyire nem voltam teljesen biztos abban, hogy mi lenne ez a megfelelő kezelési módszer.

Természetesen az alkalmazás kapcsán rengeteg ötletem van, amelyet szeretnék még megvalósítani, melyekből néhány felsorolásra is került a „Lehetőségek az alkalmazás továbbfejlesztésére” részlegnél, viszont a legfontosabbnak azt tartom, hogy összességében sikerült egy jól működő, könnyen kezelhető webalkalmazást létrehoznom, amely teljesíti a fő kitűzött célokat, az eddigi fejlesztés során szerzett tapasztalatokat pedig garantáltan a továbbfejlesztés iránt is fogom kamatoztatni.

7. Irodalomjegyzék

[1] **React.dev:** React Official Documentation

Elérhető: https://react.dev/

[2] **Mozzila Developer Network (MDN):** SPA (Single-page application)

Elérhető: https://developer.mozilla.org/en-US/docs/Glossary/SPA

[3] **React.dev:** Writing Markup with JSX

Elérhető: https://react.dev/learn/writing-markup-with-jsx

[4] **React.dev:** Built-in React Hooks

Elérhető: https://react.dev/reference/react/hooks

[5] **Reactrouter.com:** A user-obsessed, standards-focused, multi-strategy router you can deploy anywhere.

Elérhető: https://reactrouter.com/

[6] **HTML:** Learn HTML Code, Tags & CSS

Elérhető: https://html.com/

[7] **Mozilla Developer Network (MDN):** Cascading Style Sheets

Elérhető: https://developer.mozilla.org/en-US/docs/Web/CSS

[8] **Adobe Developer:** CSS modules

Elérhető: https://developer.adobe.com/commerce/pwa-studio/guides/general-concepts/css-modules/

[9] **Mozilla Developer Network (MDN):** JavaScript

Elérhető: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[10] **Oracle:** Java Platform, Standard Edition (Java SE)

Elérhető: https://docs.oracle.com/en/java/

[11] **IBM:** Java Virtual Machine

Elérhető: https://docs.oracle.com/en/java/javase/23/vm/java-virtual-machine-technology-overview.html

[12] **Spring.io:** Spring Boot

Elérhető: https://spring.io/projects/spring-boot

[13] **Spring.io:** Building REST services with Spring

Elérhető: https://spring.io/guides/tutorials/rest

[14] **Spring.io:** Building an Application with Spring Boot

Elérhető: https://spring.io/guides/gs/spring-boot

[15] **Spring.io:** Spring Security

Elérhető: https://spring.io/projects/spring-security

[16] **Spring.io:** Spring Data JPA

Elérhető: https://spring.io/projects/spring-data-jpa

[17] **IBM:** Java Persistence API (JPA)

Elérhető: https://www.ibm.com/docs/en/was-liberty/nd?topic=liberty-java-persistence-api-jpa

[18] **Hibernate:** Your relational data. Objectively.

Elérhető: https://hibernate.org/orm/

[19] **Apache Maven Project:** Maven

Elérhető: https://maven.apache.org/guides/

[20] **Apache Maven Project:** Introduction to the Dependency Mechanism

Elérhető: https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html

[21] **Project Lombok:** Lombok

Elérhető: https://projectlombok.org/

[22] **ModelMapper:** Simple, Intelligent, Object Mapping.

Elérhető: https://modelmapper.org/

[23] **Jwt.io:** JSON Web Token

Elérhető: https://jwt.io/introduction

[24] **Spring.io:** Validation

Elérhető: https://docs.spring.io/spring-boot/reference/io/validation.html#io.validation

[25] **MySQL:** The world's most popular open source database

Elérhető: https://dev.mysql.com/doc/

[26] **IBM:** What is structured query language?

Elérhető: https://www.ibm.com/think/topics/structured-query-language

[27] **Axios:** Promise based HTTP client for the browser and node.js

Elérhertő: https://axios-http.com/

[28] **Mozzila Developer Network (MDN):** HTTP

Elérhető: https://developer.mozilla.org/en-US/docs/Web/HTTP

[29] **Json.org:** Introducing JSON

Elérhető: https://www.json.org/json-en.html

[30] **GitHub:** GitHub Docs

Elérhető: https://docs.github.com/en

[31] **Git:** --distributed-even-if-your-workflow-isnt

Elérhető: https://git-scm.com/

[32] **ChatGPT:** ChatGPT

Elérhető: https://chatgpt.com/

[33] **Mozzila Developer Network (MDN):** MVC

Elérhető: https://developer.mozilla.org/en-US/docs/Glossary/MVC

[34] **Microsoft:** Description of the database normalization basics

Elérhető: https://learn.microsoft.com/en-us/office/troubleshoot/access/database-normalization-description

[35] **Mozzila Developer Network (MDN):** UUID

Elérhető: https://developer.mozilla.org/en-US/docs/Glossary/UUID

[36] **Postman:** AI is powered by APIs. APIs are powered by Postman.

Elérhető: https://www.postman.com/

8. Köszönetnyilvánítás

Eme néhány szó keretein belül szeretném kifejezni hálámat és köszönetemet témavezetőmnek, Major Sándor Rolandnak, amiért elvállalta a szakdolgozatom vezetését, illetve hasznos tanácsaival segítette annak létrejöttét. Külön köszönettel illetem szüleimet, akik a hosszú éveken át tartó munkájukkal és mérhetetlen támogatásukkal lehetővé tették számomra, hogy mindez megvalósulhasson, illetve barátaimat, hallgatótársaimat, akik segítettek a nehezebb időszakok átvészelésében.